codex-windows / 019f44c1-a285-7a30-b8d1-33c04702871f

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\sessions\2026\07\09\rollout-2026-07-09T08-12-48-019f44c1-a285-7a30-b8d1-33c04702871f.jsonl`
- SHA-256: `e1c0c4d73f8d12e291334bd785695d16e75a29b9669f7c8947f47c96ec773f93`
- Source modified: `2026-07-09T05:34:38+00:00`
- Imported at: `2026-07-09T08:29:36+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\khelsutra-guru`
- model_provider: `openai`
- session_id: `019f44c1-a285-7a30-b8d1-33c04702871f`
- source: `vscode`

Transcript

1. developer (2026-07-09T02:43:35.963Z)

<permissions instructions>
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled. Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason, commands will be rejected.
</permissions instructions>
<app-context>
Codex desktop context
- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files
- In the app, the model can display images and videos using standard Markdown image syntax: ![alt](url)
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., ![alt](/absolute/path.png)); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., [label](https://example.com)).

Workspace Dependencies
- For sheets, slides, and documents, call `load_workspace_dependencies` to find the bundled runtime and libraries.

Automations
- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the `automation_update` tool first, then follow its schema instead of writing raw automation directives by hand.
- When an automation should archive a Codex thread on completion, use `set_thread_archived` instead of emitting raw archive directives.

Thread Coordination
- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: `create_thread`, `fork_thread`, `list_threads`, `read_thread`, `send_message_to_thread`, `handoff_thread`,

`set_thread_pinned`, `set_thread_archived`, or `set_thread_title`.
- Only use `create_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments
- Use the ::code-comment{...} directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: ::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}
</app-context>
<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>
<apps_instructions>
Apps (Connectors)
Apps (Connectors) can be explicitly triggered in user messages in the format `[$app-name](app://{connector_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.
An app is equivalent to a set of MCP tools within the `codex_apps` MCP.
An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool_search` tool. If `tool_search` is available, the apps that are searchable by `tools_search` will be listed by it.
Do not additionally call list_mcp_resources or list_mcp_resource_templates for apps.
</apps_instructions>
<skills_instructions>
Skills
A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.
Skill roots
- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`

- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.7-2841cf9749ae/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.5/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.4/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.8/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.4/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into $CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting (file: r2/pdf/SKILL.md)
- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)
- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggeri (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting,

cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.141536/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.141536/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.141536/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)
- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)

- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)
- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or canc (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple

people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and referenc (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)
- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)
- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)
- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-summarization/SKILL.md)
- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)
- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)
- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using `slack_send_message`, `slack_send_message_draft`, or `slack_create_canvas`. Use `slack` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-

message/SKILL.md)
- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes $template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
  1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
  2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
  3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
  4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
  5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
  - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
  - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
  - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
  - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
  - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.
</skills_instructions>
<plugins_instructions>

Plugins

A plugin is a local bundle of skills, MCP servers, and apps.

How to use plugins

- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp__server__tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the

plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-09T02:43:35.963Z)

AGENTS.md instructions

<INSTRUCTIONS>
Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading

Always `git pull` a repo before starting to read it ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- **Why:** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- **How (safe form):** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- **If it can't fast-forward** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts

Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit **concurrently**, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a **squash-merge silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:
- **Branch from the REMOTE, explicitly:** `git fetch origin` then `git checkout -b <name> origin/<base>`
  (e.g. `origin/master`). Never assume the current branch is the intended base.
- **Re-verify right before `git add`/commit:** `git branch --show-current` + `git log --oneline -1`
  (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your
  commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- **Stage explicit paths** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked
  files can't ride into your commit.
- **Serialize repo writes:** don't start a repo-writing spawned/background task while you hold
  uncommitted work ? commit or push first. If one may be running, treat the tree + current branch
  as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living **session handoff** for each project and use it to resume context quickly across windows.

- **Where it lives:**
  - If the project has an auto-memory dir (`~/.Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a **"? Start here"** entry at the top of that project's `MEMORY.md`.
  - Otherwise, keep a `SESSION_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
  1. **You are here** ? current state.

  2. **Next steps / open threads.**
  3. **Ramp-up kit** ? the exact files/docs to read to get current.
  4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it
  (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is
  pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a
  restart can ramp up without losing context. Do **not** rewrite it every turn ? only when
  something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the
  handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's
  attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally
verified the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a *verified* stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.
</INSTRUCTIONS>
<environment_context>
  <cwd>C:\Users\avidu\Projects\khelsutra-guru</cwd>
  <shell>powershell</shell>
  <current_date>2026-07-09</current_date>
  <timezone>Asia/Calcutta</timezone>
  <filesystem><workspace_roots><root>C:\Users\avidu\Projects\khelsutra-guru</root></workspace_roots><permission_profile type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>

3. user (2026-07-09T02:43:36.003Z)

can you look at the sports-data-vault repo and ensure its in sync with remote head and identify how can I set it up on this machine. F: and G: drivers are connected so do a thorough study and make a plan to onboard it

4. assistant (2026-07-09T02:43:48.698Z)

I?ll first bring the relevant checkouts up to remote with `--ff-only`, then I?ll read the repo?s handoff/docs and inspect how the external drives fit into setup. After that I?ll turn it into an onboarding plan you can actually run on this machine.

5. assistant (2026-07-09T02:44:00.356Z)

The current folder isn?t itself a Git checkout, so there was nothing to pull there. I?m going to locate `sports-data-vault` directly across the local projects area and the connected drives, then I?ll fast-forward that repo before reading anything inside it.

6. assistant (2026-07-09T02:46:11.251Z)

The exact `sports-data-vault` directory didn?t show up in the quick pass before the wide drive scan timed out. I did see `sports-data-collector` clones, so I?m narrowing the search by directory depth and Git remotes to distinguish ?vault? from similarly named repos.

7. assistant (2026-07-09T02:46:37.442Z)

I?ve got three likely repo-like candidates now: a local `sports-data-collector`, two synced-drive copies of it, and a `rally-corpus-vault` under `G:`. I?m fast-forwarding each candidate before opening docs or code so the setup plan is based on current remote state wherever possible.

8. assistant (2026-07-09T02:48:37.804Z)

The synced-drive copies of `sports-data-collector` were behind and are now fast-forwarded to `c5b24bc`; the local C: copy was already current. I?m checking whether there is a distinct `sports-data-vault` remote, then I?ll study the current repo docs/config and the F:/G: data layout.

9. assistant (2026-07-09T02:49:04.305Z)

There doesn?t appear to be a separate accessible `sports-data-vault` GitHub repo under the obvious org/user names; the repo on disk that matches the sports-data work is `sports-data-collector`. One G: copy has an uncommitted database file after sync, so I?ll treat the clean C: checkout as the canonical onboarding target and still account for the drive data in the plan.

10. assistant (2026-07-09T02:49:20.464Z)

The clean checkout is confirmed at the same commit as `origin/master`: `c5b24bc`, so remote sync is good for the C: repo. I don?t see a repo-local handoff file, so I?m checking the Codex project memory location before diving into README/docs/config.

11. assistant (2026-07-09T02:49:35.689Z)

The repo is a small Python CLI/package: SQLite metadata, curated annotations, optional video fetching, and export into the highlight-indexer?s eval/training format. I?m reading the pipeline docs and config next, then I?ll verify what this Windows machine already has installed versus what it still needs.

12. assistant (2026-07-09T02:49:49.909Z)

The golden corpus manifest explicitly expects both drives: F: for owned GoPro originals and G: for annotation proxies / Drive-shared labeled videos. I?m now checking local prerequisites and whether the repo can run as-is, before I recommend any setup steps.

13. assistant (2026-07-09T02:50:02.840Z)

The machine already has the heavy external pieces: Python, FFmpeg/ffprobe, Deno, yt-dlp, and Git LFS. The CLI imports and reports `8 curated / 23 staging` badminton records, so I?m going one level deeper now: tests/metadata, current video path availability, and how the F:/G: corpus maps to the manifest.

14. assistant (2026-07-09T02:50:22.377Z)

Test suite mostly runs: `304 passed`, but two tests currently assume `obsreport` is absent while this machine has it importable, so they fail. That?s useful onboarding signal: use a clean repo venv for default tests, and install the optional report extra only when you intentionally want report emission.

15. assistant (2026-07-09T02:50:39.020Z)

The failure is clearly environmental rather than a broken CLI path: the tests were written for the ?soft dependency absent? path, but this global Python can see `obsreport`. I?ll avoid changing code and keep using this as setup guidance; next I?m checking the corpus manifest paths

against the attached drives.

16. assistant (2026-07-09T02:51:01.683Z)

A couple of my PowerShell one-liners tripped over pipeline syntax, so I?m rerunning those checks in a cleaner shape. The important bit already surfaced: `obsreport` is being imported from the sibling `sports-obsreport` checkout, which explains the two test failures.

17. assistant (2026-07-09T02:51:48.741Z)

The DB is healthy enough to resolve most local paths: all six F: ?owned original? curated videos exist, and the ShuttleSet staging videos mostly exist under repo `data/videos`. The export command may hash large files and exceeded my short timeout, so I?m checking the implementation and then I?ll run it with a realistic timeout if it?s safe.

18. assistant (2026-07-09T02:51:54.174Z)

Export intentionally full-hashes source videos because the downstream indexer joins by file MD5. That explains the timeout: the commercial curated set includes multi-GB F: originals, so a real export can take a few minutes even though the command is working.

19. assistant (2026-07-09T02:53:21.877Z)

The real export succeeded in 71 seconds and produced 7 commercial badminton eval videos / 264 segments. One DB row, `my_custom_local_match`, is a placeholder with a missing local video, so it exports without MD5 and should be cleaned or excluded before downstream indexer work.

20. assistant (2026-07-09T02:53:36.786Z)

I also found a real `rally-corpus-vault` repo on G:, and it is synced to `origin/master` at `6785afd`. Since your wording said ?vault,? I?m studying that repo too; it looks like the storage/control-plane piece that should sit beside `sports-data-collector`, not a duplicate of it.

21. assistant (2026-07-09T02:53:49.937Z)

The vault repo is deliberately pure-stdlib Python with a `storagectl` console script, and the per-machine piece is `vault.local.json` pointing at F:, G:, and GCS. I?m checking whether those configured roots/tools already exist and whether the vault test suite passes here.

22. assistant (2026-07-09T02:54:04.181Z)

The vault is synced and its tests pass (`29 passed`). Setup gap is concrete: `vault.local.json` does not exist, the expected F: corpus root does not exist yet, G: corpus root exists but is empty, and `gcloud` is installed while `rclone`/`rsync` are not.

23. assistant (2026-07-09T02:54:13.212Z)

One nuance: `storagectl` takes global flags before the subcommand, so my quick `status` invocation had the flags in the wrong place. I?m correcting that and sampling the seed manifest format.

24. assistant (2026-07-09T02:54:30.931Z)

I?ve finished the study pass. I?m doing one last status check so I can separate ?repo is synced? from any local test/cache noise and avoid mixing the two in the final plan.

25. assistant (2026-07-09T02:55:12.332Z)

I couldn?t find an exact repo named `sports-data-vault` on C:/F:/G: or via the obvious GitHub search. What I found is likely the intended pair:

- [sports-data-collector](</C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector>)

synced to `origin/master` at `c5b24bc`, clean.
- [rally-corpus-vault](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault>) synced to `origin/master` at `6785afd`, clean.

There are also two G: copies of `sports-data-collector`; one is clean, one has a local modified `data/sports_metadata.db`, so I would treat the clean C: checkout as the working repo.

What I Verified
`F:` and `G:` are connected. The collector?s golden manifest expects exactly those drives:

- F: owns GoPro originals, including the six curated local golden videos.
- G: holds annotation proxies / Drive-shared labeled videos.
- `G:\My Drive\KhelSutraGuru\Sports Videos Corpus` exists but is currently empty.
- The vault?s expected F: corpus root, `F:\[Khelsutra] GoPro Hero Black 12\corpus`, does not exist yet.

Machine prerequisites look good for collector work: Python, FFmpeg/ffprobe, Deno, yt-dlp, Git LFS, and gcloud are installed. Missing for vault-style broader syncing: `rclone` and `rsync`.

Collector health:

- `python -m src.cli.curate status` works: `8 curated`, `23 staging`.
- Export works and took 71s because it full-hashes multi-GB F: videos.
- Export produced `7` commercial videos / `264` segments.
- Warning: `my_custom_local_match` points to missing `data/videos/my_custom_match.mp4`, so it exports without MD5. That should be removed/fixed before downstream indexer use.
- Tests: `304 passed, 2 failed`. The two failures are environmental: tests assume `obsreport` is absent, but this machine imports it from sibling `sports-obsreport`. Run tests in a clean venv before installing/report-linking `obsreport`.

Recommended Onboarding Plan
1. Use the clean collector checkout:
```powershell
cd C:\Users\avidu\Projects\khelsutra-guru\sports-data-collector
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -U pip
pip install -e ".[fetch,dev]"
$env:PYTHONPATH="."
python -m src.cli.curate status
python -m pytest tests
python -m ruff check .
```

2. Install optional reporting only after the clean test baseline:
```powershell
pip install -e ..\sports-obsreport --no-build-isolation
```

3. Treat the vault as the storage custody layer:
```powershell
cd "G:\Other computers\My Laptop (1)\Projects\khelsutra-guru\rally-corpus-vault"
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
copy vault.example.json vault.local.json
python -m pytest tests
```

4. Bootstrap the vault deliberately:
Create/confirm these tier roots first:

```text
F:\[Khelsutra] GoPro Hero Black 12\corpus
G:\My Drive\KhelSutraGuru\Sports Videos Corpus
```

```

Then add golden/ratified videos into the canonical layout with `storagectl add`, copying from the existing GoPro/annotation source paths into `videos/full`. Do this before any dedup/delete work.

5. After bootstrap:
```powershell
storagectl status
storagectl verify F
storagectl verify G
```

6. For GCS staging, use `storagectl stage`/`gcloud`, but avoid raw bracketed F: paths with `gcloud storage cp`; the vault docs call out that `[Khelsutra]` is a glob hazard.

Bottom line: the code repos are synced. The collector is runnable now. The vault needs local config plus an intentional first corpus bootstrap into the normalized F:/G: layout.

26. user (2026-07-09T02:55:12.876Z)

rally-corpu-vault is hte right one which I meant

27. assistant (2026-07-09T02:55:30.006Z)

Got it, `rally-corpus-vault` is the right target.

Current state: [rally-corpus-vault](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault>) is synced cleanly to `origin/master` at `6785afd`. Its test suite passes: `29 passed`.

Onboarding blockers/gaps I found:

- `vault.local.json` is missing, so `storagectl` has no machine-specific tier config yet.
- Expected F: corpus root does not exist yet: `F:\[Khelsutra] GoPro Hero Black 12\corpus`
- G: corpus root exists but is empty: `G:\My Drive\KhelSutraGuru\Sports Videos Corpus`
- `gcloud` is installed.
- `rclone` and `rsync` are not installed, but they are only needed for some sync paths.
- F: currently has the raw GoPro/source folders, not the normalized vault layout.

Best onboarding path:

1. Create `vault.local.json` from `vault.example.json`.
2. Create the F: corpus root and canonical subfolders: `videos/full`, `videos/proxy`, `derived/labels`, `derived/trajectories`, `manifests`.
3. Use `storagectl add` to promote golden/ratified videos from existing F:/G: source locations into the normalized F:/G: corpus layout.
4. Run `storagectl status`, then `storagectl verify F` and `storagectl verify G`.
5. Only after both durable tiers verify, consider dedup/quarantine or GCS staging.

I would treat `sports-data-collector` as the metadata/label producer, and `rally-corpus-vault` as the durable storage source of truth.

28. user (2026-07-09T02:57:52.904Z)

Can you write these as the onboarding instructions and also identify how to merge this metadata across different machines or this is for a single machine setup only. After that I would like to do all the data moves and organize in this format

29. assistant (2026-07-09T02:58:03.503Z)

I?ll put this into the vault repo as durable onboarding docs, then I?ll call out the metadata model: what is portable across machines, what stays local, and how to reconcile manifests safely before we start moving bytes.
```

30. assistant (2026-07-09T02:58:15.725Z)

The repo is already up to date and there?s no existing handoff/onboarding doc. I?m going to add a focused onboarding guide and link it from the README, keeping `vault.local.json` machine-specific and the corpus manifest as the shared artifact.

31. assistant (2026-07-09T02:58:25.529Z)

I?m going to make three scoped doc edits: add `ONBOARDING.md`, link it from the README, and fix the ignore rule so `vault.local.json` stays local as the docs already intend. No data moves yet.

32. assistant (2026-07-09T02:59:22.179Z)

The first draft is in place. I noticed one small operational detail in the metadata recovery step: the repo `manifests\` directory may not exist before the first write, so I?m tightening that instruction before I run the checks.

33. assistant (2026-07-09T02:59:27.552Z)

Tests and lint are clean. I?m applying that small doc correction now, then I?ll give you the exact files and the metadata/merge answer in short form.

34. assistant (2026-07-09T02:59:36.622Z)

Since this repo didn?t have a handoff file, I?m also adding a concise `SESSION_HANDOFF.md` so the next window can start from the current onboarding state instead of rediscovering it.

35. assistant (2026-07-09T03:00:04.304Z)

Done. I added the onboarding instructions into the vault repo and kept it focused on setup before data moves.

Changed files:

- [ONBOARDING.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/ONBOARDING.md>) with machine snapshot, setup steps, F:/G: corpus folder creation, safe data-move order, and multi-machine metadata workflow.
- [README.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/README.md>) now points to the onboarding guide and clarifies portable vs local metadata.
- [.gitignore](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/.gitignore>) now ignores `vault.local.json`.
- [SESSION_HANDOFF.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/SESSION_HANDOFF.md>) so future sessions start from the current state.

Key answer: this is not single-machine only. `vault.local.json` is per-machine, but `manifests/corpus_index.json` is the shared portable metadata. The current merge model is additive and git-like, not a conflict-free database: pull first, avoid concurrent metadata writers, merge by `video_id`/`canonical_name`, and verify bytes with `storagectl verify` after any reconciliation.

Verification passed:

```text
python -m pytest tests -> 29 passed
python -m ruff check . -> clean
```

No data moves were performed yet.

36. user (2026-07-09T03:07:26.516Z)

can you map out the steps needed, smoke tests and other whcih need to be run to populate my vault right now?

37. assistant (2026-07-09T03:07:38.270Z)

I?ll turn this into a concrete population runbook for the current F:/G: machine state: preflight checks, first safe copy batch, smoke tests after each stage, and the gates before dedup or cloud staging. I?ll refresh the repo first, then I?ll derive commands from the vault tool and the known golden source paths.

38. assistant (2026-07-09T03:08:09.692Z)

I?ve got the existing onboarding/spec context. Next I?m checking the current CLI argument surface and whether all 15 golden source paths from the seed actually exist on this Windows mount, so the runbook can be executable rather than vague.

39. assistant (2026-07-09T03:08:24.995Z)

All 15 golden source videos exist on F: and total about 111.87 GB, so F: and G: have enough reported free space for the initial durable-copy bootstrap. I?m adding a population runbook with a small first-video smoke test, then the full batch commands and post-move verification gates.

40. assistant (2026-07-09T03:10:07.258Z)

The runbook is written. I spotted one shell-style Python snippet that doesn?t belong in a PowerShell-first doc, so I?m cleaning that out and then I?ll run the repo checks again.

41. assistant (2026-07-09T03:10:18.719Z)

The PowerShell runbook is now internally consistent. I?m running the project tests/lint and checking the final diff so we can use this as the checklist for the actual moves.

42. assistant (2026-07-09T03:10:34.753Z)

Mapped it out as a concrete runbook here:

[POPULATE_VAULT_RUNBOOK.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/POPULATE_VAULT_RUNBOOK.md>)

It includes:

- Preflight: repo sync, venv setup, tests, ruff, tool/free-space checks.
- Local config: create `vault.local.json` and canonical F:/G: folder layout.
- Source smoke test: confirms all 15 golden source videos exist on F: and total about `111.87 GB`.
- First data smoke: copy only `mahadevpura_singles` to F and G, then `storagectl status`, `verify F`, `verify G`.
- Full population: loop to add the remaining 14 golden videos with `storagectl add ... --to F,G`.
- Post-population gates: expect 15 unique videos, 0 N+1 violations, 15 ok on both F and G, and matching manifest hashes.
- Stop conditions: copy verify failure, hash mismatch, missing source, duplicate canonical naming, Drive sync/quota issues.
- Later-only work: dedup, GCS staging, proxies.

I also linked it from [README.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/README.md>) and updated [SESSION_HANDOFF.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/SESSION_HANDOFF.md>) so the next session starts with this exact plan.

Verification still passes:

```text
python -m pytest tests -> 29 passed
python -m ruff check . -> clean
```

No data moves were run yet.

43. user (2026-07-09T03:10:35.342Z)

make sure to identify the gaps in the vault's instructions and features as you do this exerciuse and report back with bugs or PRs to improve the repo for others

44. assistant (2026-07-09T03:10:51.652Z)

Absolutely. I?ll treat the population runbook as a usability test of the repo itself: every point where the docs, CLI, or safety model forces tribal knowledge will become a concrete bug/PR candidate. I?ll inspect the current implementation against the runbook and then capture the improvement backlog in-repo plus summarize the highest-priority fixes.

45. assistant (2026-07-09T03:11:31.449Z)

The first pass surfaced several real repo gaps, not just nice-to-haves. The biggest pattern is that the docs describe the end-state tool, while the current v0.1 implementation only fully wires `add`, `index`, `status`, `verify`, `dedup`, and `evict`; `sync`, `stage`, and `proxy` are still stubs. I?m going to capture these as prioritized PR candidates with concrete failure modes.

46. assistant (2026-07-09T03:11:51.039Z)

I confirmed two concrete behavioral bugs with tiny temp files: `add --to NOPE` writes an unbacked manifest entry, and adding F then G separately leaves a correct merged manifest but returns failure on the second add because N+1 is computed from only the current command?s tier set. I?m checking the collision behavior too, because that?s the one that could silently poison a manifest.

47. assistant (2026-07-09T03:12:21.599Z)

The collision check confirms another safety issue: `storagectl add` will overwrite an existing canonical name with a different file and rewrite the manifest without requiring an explicit replace. That may be useful someday, but for corpus custody it should be a hard stop unless the operator says ?replace this identity? very deliberately.

48. assistant (2026-07-09T03:13:20.604Z)

I?ve added a formal improvement backlog and linked it from the README/session handoff. I?m running the verification suite one more time so the docs-plus-backlog changes don?t leave the repo in a questionable state.

49. assistant (2026-07-09T03:13:40.711Z)

Done. I added a repo-level backlog here:

[IMPROVEMENT_BACKLOG.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/IMPROVEMENT_BACKLOG.md>)

I also linked it from [README.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/README.md>) and updated [SESSION_HANDOFF.md](</G:/Other computers/My Laptop (1)/Projects/khelsutra-guru/rally-corpus-vault/SESSION_HANDOFF.md>).

Top gaps found while exercising the population flow:

1. `storagectl add --to NOPE` can write a manifest entry with no verified tier copies.
2. Incremental add has a bad exit code: add to `F`, then add to `G`, and the merged manifest is correct but the second command still returns failure because N+1 is computed from the current command only.
3. `add` allows same `canonical_name` with different bytes, overwriting the tier file and manifest without an explicit replace guard.
4. `dedup --quarantine/--delete` is not actually manifest/N+1 guarded, despite the docs implying guarded deletion.

5. `sync`, `stage`, and `proxy` are documented like working commands, but are stubs or do not parse the documented examples.
6. Missing `vault.local.json` should produce a clear setup error, not empty/odd behavior.
7. The repo needs first-class `preflight`, `validate-manifest`, and batch population commands so future operators do not need a long hand-written PowerShell array.

Suggested PR order is in the backlog: harden `add` first, fix doc/CLI truth for stubs, add `preflight` and manifest validation, then batch-add, then dedup guards.

Verification still passes:

```text
python -m pytest tests -> 29 passed
python -m ruff check . -> clean
```

No data moves were run.

50. user (2026-07-09T03:14:01.292Z)

can you make all this in a PR and send for review and then we can execute the vault migrations?

51. assistant (2026-07-09T03:14:07.110Z)

Yes. I?ll package the docs/backlog work into a clean branch and draft PR, following the repo branching safety rules so none of this rides on the wrong base. First I?m loading the PR-publish workflow instructions, then I?ll commit only the intended doc files.

52. assistant (2026-07-09T03:14:21.714Z)

I?m using the GitHub publish workflow for this turn. The intended PR scope is just the vault onboarding/runbook/backlog docs plus the local-config ignore rule; I?ll keep staging explicit.

53. assistant (2026-07-09T03:14:32.401Z)

The worktree scope is clean: only the five new/modified docs plus `.gitignore`, no unrelated changes. I?m branching explicitly from `origin/master` now; the uncommitted doc edits will carry onto that branch, then I?ll re-run checks and commit.

54. assistant (2026-07-09T03:14:37.002Z)

Fetch confirms `origin/master` is still `6785afd`. I?m creating the PR branch from that remote base now.

55. assistant (2026-07-09T03:14:47.264Z)

Branch is now `agent/vault-onboarding-runbooks` based on `origin/master`. I?m doing the required pre-commit verification of branch/HEAD, then staging only the intended files.

56. assistant (2026-07-09T03:14:53.172Z)

Checks are green on the branch. I?m staging only the onboarding/runbook/backlog docs, README link, handoff, and ignore rule now.

57. assistant (2026-07-09T03:15:09.435Z)

The staged diff is exactly the PR scope: six files, all docs/ignore metadata. I?m doing the final branch/HEAD check immediately before committing, then I?ll push and open a draft PR.

58. assistant (2026-07-09T03:15:22.863Z)

Commit is created. I?m checking that the branch contains only this one commit over `origin/master`, then pushing it with upstream tracking.

59. assistant (2026-07-09T03:15:26.191Z)

The branch has exactly one commit over `origin/master`: `31745e1`. I?m pushing now, then I?ll create the draft PR with a body that calls out the safety gaps before migration.

60. assistant (2026-07-09T03:15:38.120Z)

Branch is pushed. I?m looking for the GitHub PR creation tool first; if it isn?t exposed here, I?ll use `gh pr create` as the workflow fallback.

61. assistant (2026-07-09T03:15:49.785Z)

The GitHub PR tool is available, so I?ll use it instead of falling back to `gh`. I?m fetching the repo metadata for the base branch, then opening the PR against `master`.

62. assistant (2026-07-09T03:16:09.845Z)

The GitHub connector hit a 404 creating the PR, likely a connector access quirk for this private/org repo. The branch is already pushed, so I?m using the approved `gh pr create` fallback with the same title/body.

63. assistant (2026-07-09T03:16:29.485Z)

PR #4 is open. Since that?s a meaningful checkpoint, I?m updating the session handoff with the PR link and pushing that small follow-up to the same branch.

64. assistant (2026-07-09T03:16:38.596Z)

The handoff now points to PR #4. I?m committing just that file and pushing the update to the PR branch.

65. assistant (2026-07-09T03:16:56.623Z)

The PR branch is updated with the handoff link. I?m doing a final branch/status sanity check so the close-out is exact.

66. assistant (2026-07-09T03:17:04.609Z)

PR is open and ready for review:

https://github.com/Khelsutra/rally-corpus-vault/pull/4

Branch: `agent/vault-onboarding-runbooks`
Base: `master`
Status: open, not draft

Included:

- `ONBOARDING.md`
- `POPULATE_VAULT_RUNBOOK.md`
- `IMPROVEMENT_BACKLOG.md`
- `SESSION_HANDOFF.md`
- README links
- `.gitignore` update for `vault.local.json`

Validation run before opening the PR:

```text
python -m pytest tests -> 29 passed
python -m ruff check . -> clean
```

Final branch sanity check: only our two commits are ahead of `origin/master`, and the worktree

is clean.

67. user (2026-07-09T03:18:02.780Z)

merged. Can we proceed to the data migration?

68. assistant (2026-07-09T03:18:25.417Z)

Yes. Since the PR is merged, I?ll move into execution mode: update `master`, create a fresh migration branch for the manifest metadata, run the preflight, then do the one-video smoke copy before touching the rest of the batch. I?ll avoid the known `storagectl add` edge cases by adding each video to `F,G` in one command with known-good tier names.

69. assistant (2026-07-09T03:18:33.702Z)

`origin/master` now includes the merged PR. I?m switching to `master`, fast-forwarding, then cutting a new migration branch from `origin/master` so the corpus manifest update can be reviewed separately from the data-copy operation.

70. assistant (2026-07-09T03:18:48.518Z)

The migration branch is live at `agent/bootstrap-golden-corpus`, based on merged `origin/master`. I?m running the preflight checks now: tests/lint, local config/root setup, drive space, and source-file presence.

71. assistant (2026-07-09T03:18:55.741Z)

Preflight checks are green: tests pass, lint is clean, ffprobe/gcloud are present, and both drives have enough space for the initial 112 GB batch. I?m creating the local vault config and canonical folder structure next.

72. assistant (2026-07-09T03:19:15.058Z)

Tiny snag before any data moved: the runbook command used `New-Item -LiteralPath`, but this PowerShell has no `-LiteralPath` on `New-Item`. That?s a docs bug from the dry run. I?m switching to `.NET` directory creation, which handles bracketed paths safely, and I?ll record the runbook fix after the migration checkpoint.

73. assistant (2026-07-09T03:19:32.595Z)

The empty-vault smoke test passes: `0 unique videos`, no N+1 violations. The display command also reminded us why bracketed F: paths need `-LiteralPath` for reads; I?ll use literal path checks for the source smoke now.

74. assistant (2026-07-09T03:19:58.132Z)

Source smoke passed: all 15 planned golden files exist and total `111.87 GB`. I?m doing the one-video data smoke with the smallest source, `mahadevpura_singles`, copying it into both durable tiers and verifying both copies.

75. assistant (2026-07-09T03:20:28.927Z)

The one-video smoke passed cleanly: add exit `0`, both `verify F` and `verify G` report `1 ok`, and N+1 is satisfied. I?m starting the remaining 14-video batch now in a logged background PowerShell process so I can poll progress instead of leaving us blind during the long copy/hash cycle.

76. assistant (2026-07-09T03:21:02.948Z)

The batch process is running as PID `24216`, logging under `scratch/bootstrap_remaining_golden_20260709_085058.log`. I?ll poll it until it finishes, then run independent final verification before committing metadata.

77. assistant (2026-07-09T03:21:20.863Z)

The background process exited immediately without writing its log, so it likely failed before entering the script body. No remaining data moves happened from that batch process. I?m inspecting the generated script and will rerun the batch foreground or with a simpler invocation once the issue is clear.

78. assistant (2026-07-09T03:21:35.353Z)

The script itself is valid; the first background launch likely lost the quoted `-File` path because it contains spaces. I?m restarting it with explicit quoting around the script path, still writing to the same log/done files.

79. assistant (2026-07-09T03:21:47.754Z)

The batch is now running as PID `25380`. I?ll poll the log; each `ADD_DONE` means that video has been copied to both durable tiers and the manifest/status check after it passed.

80. user (2026-07-09T03:21:53.877Z)

can you add this copy operation in the vault as a command and ensure it logs something every 30s as a note of progress

81. user (2026-07-09T03:21:54.062Z)

the entire process should be driven by the vault and not need any intervention

82. assistant (2026-07-09T03:22:51.149Z)

Good call. The failed background wrapper is exactly the kind of thing the vault should own. I?m pausing manual migration and switching to a code PR: add a first-class `storagectl batch-add` driven by a committed plan, with internal progress logging every 30 seconds, plus hardening so it does not need wrapper scripts or operator babysitting.

83. assistant (2026-07-09T03:22:57.624Z)

There?s one untracked repo manifest from the successful one-video smoke test; I?ll leave it untouched and uncommitted. I?m creating a fresh code branch from `origin/master` and will stage only source/docs/tests for the new vault-driven migration command.

84. assistant (2026-07-09T03:23:11.305Z)

I?m going to implement this as a general `batch-add` command rather than a one-off script: it will read a CSV plan, validate targets/sources up front, copy via the existing vault backends, and emit timestamped progress to stdout plus an optional log file.

85. assistant (2026-07-09T03:23:28.577Z)

The progress logger and progress-aware hashing are in place. Next I?m wiring filesystem copies through a progress-aware copy-to-temp-and-verify path so long local copies report liveness and avoid leaving half-written destination files as final outputs.

86. assistant (2026-07-09T03:23:43.871Z)

Now I?m adding the shared copy helper and refactoring `add` into a reusable function so both single-video and batch-video flows get the same validation, copy verification, and N+1 result handling.

87. assistant (2026-07-09T03:24:46.513Z)

The single-add path now has target validation, collision guards, and merged-manifest N+1 calculation. I?m adding `batch-add` on top of that, plus a committed golden bootstrap CSV so the migration can be started with one vault command.

88. assistant (2026-07-09T03:25:19.427Z)

One important migration resilience detail: the batch command should persist the manifest after each successful video, not only at the end. I?m making that the default so an interruption leaves already-verified copies recorded and resumable.

89. assistant (2026-07-09T03:25:44.815Z)

The CLI now has `batch-add`. I?m adding a committed CSV plan for the current 15-video golden bootstrap so the migration command can be a single vault-owned invocation rather than a hand-maintained shell loop.

90. assistant (2026-07-09T03:26:04.125Z)

The plan file is in place. I?m adding tests around the new command and the safety hardening so this doesn?t regress: batch success, missing source preflight, unknown tiers, incremental N+1, and same-name identity collision.

91. assistant (2026-07-09T03:26:43.899Z)

The new command passes tests (`34 passed`) and lint. I?m updating the runbook so the actual migration is a single `storagectl batch-add ...` command with a progress log, and fixing the earlier `New-Item -LiteralPath` docs bug.

92. assistant (2026-07-09T03:27:26.110Z)

The main runbook now uses `batch-add`; I?m also fixing the same directory-creation snippet in onboarding so future setup doesn?t hit the `New-Item -LiteralPath` issue we just discovered.

93. assistant (2026-07-09T03:27:35.840Z)

I?m updating the session handoff too: it should point the next run at `storagectl batch-add`, not the old ad hoc PowerShell loop.

94. assistant (2026-07-09T03:28:09.162Z)

The implementation is green locally: `34 passed` and ruff clean. I?m going to package this as a separate PR before running the full migration, because it changes vault behavior and fixes several P0 safety issues in `add`. I?ll stage only code/docs/tests/plan, leaving the one-video smoke manifest uncommitted.

95. assistant (2026-07-09T03:28:13.185Z)

The branch is still based cleanly on `origin/master`, with no commits yet. I?m staging the intended implementation files explicitly; `manifests/` stays out of this PR.

96. assistant (2026-07-09T03:28:25.942Z)

The staged diff is the new command, tests, the golden plan CSV, and docs updates only. I?m doing the required final branch/HEAD check, then committing and opening a PR.

97. assistant (2026-07-09T03:28:37.352Z)

Committed `7b04639` with the vault-driven batch command. I?m pushing and opening a PR now; the untracked smoke-test manifest is still local only.

98. assistant (2026-07-09T03:28:45.813Z)

Branch is pushed. I?ll use `gh pr create` again since the connector had access trouble on this repo last time.

99. assistant (2026-07-09T03:29:06.494Z)

PR #5 is open for the vault-owned batch command. I?m updating the handoff with that PR link and then I?ll stop before running the full migration until you merge/review this behavior change.

100. assistant (2026-07-09T03:29:43.720Z)

Paused the full migration and opened the code PR for the vault-owned copy flow:

https://github.com/Khelsutra/rally-corpus-vault/pull/5

What it adds:

- `storagectl batch-add <plan.csv>`
- progress notes every 30s during hash/copy/verify phases
- optional `--log scratch/bootstrap_golden.log`
- committed plan: `seed/golden_bootstrap_plan.windows.csv`
- manifest persisted after each successful video for resumability
- hardening for `add`: unknown tiers fail early, no-copy failures don?t write manifest entries, incremental F then G add returns correctly, and same-name/different-MD5 collisions are blocked before overwrite
- docs updated so migration is driven by the vault command, not a PowerShell loop

Validation:

```text
python -m pytest tests -> 34 passed
python -m ruff check . -> clean
```

Current local data state: only the one-video smoke copy (`mahadevpura_singles`) has been migrated to F/G, and the repo manifest for that is still local/uncommitted under `manifests/`. I did not run the remaining 14-video migration after your request to make this vault-driven first.

101. user (2026-07-09T03:36:31.060Z)

does the cli have a smoke test mode and can you use it to identify that the changes are WAI?

102. assistant (2026-07-09T03:36:47.097Z)

There isn?t a dedicated `storagectl smoke-test` command yet. The closest built-in is global `--dry`, but for this change the best WAI check is a tiny end-to-end vault run against temporary F/G tiers: batch plan, copy, verify, rerun idempotently, and inspect the progress log without touching the real corpus.

103. assistant (2026-07-09T03:37:05.927Z)

`batch-add` is present and exposes `--progress-interval` plus `--log`, which gives us the progress-beacon behavior. My parallel status call timed out oddly, so I?m rerunning status separately and then using a temp mini-vault for the smoke.

104. assistant (2026-07-09T03:37:27.222Z)

We?re on the PR branch and the only untracked item is the local real-corpus manifest from the one-video smoke copy. I?m leaving that untouched and running the CLI smoke in `%TEMP%`.

105. assistant (2026-07-09T03:38:04.268Z)

The temp CLI smoke passed, but you?re right to ask for a real mode. There is no dedicated `smoke-test` subcommand yet, so I?m adding one to PR #5 now: it will create a temporary mini-vault, run `batch-add`, verify both tiers, rerun idempotently, and exit non-zero if anything is off.

106. assistant (2026-07-09T03:38:39.246Z)

The new `smoke-test` subcommand is wired. I?m adding a regression test and then I?ll run the real CLI smoke mode against the branch.

107. assistant (2026-07-09T03:38:46.216Z)

I?m documenting `smoke-test` in the README/runbook so reviewers and future operators can use it before touching real corpus bytes.

108. assistant (2026-07-09T03:39:00.393Z)

Now I?m running the full validation stack: unit tests, lint, CLI help, and the actual `storagectl smoke-test` command.

109. assistant (2026-07-09T03:39:13.250Z)

The new `storagectl smoke-test` passes and exercises the exact behavior we care about: batch-add, per-tier copy/verify, status, verify F/G, idempotent rerun, and progress-log markers. I noticed the smoke output includes harmless ffprobe warnings because the test files are synthetic bytes with `.mp4`; I?m silencing ffprobe stderr since metadata probing is explicitly best-effort.

110. assistant (2026-07-09T03:39:25.720Z)

I?m rerunning smoke/test/lint after silencing the best-effort ffprobe noise, then I?ll amend PR #5 so the smoke mode is part of the vault-owned migration command set.

111. assistant (2026-07-09T03:39:37.364Z)

`storagectl smoke-test` now passes with clean vault-level output and no ffprobe noise. I?m amending PR #5 with this smoke-test command and pushing the update.

112. assistant (2026-07-09T03:40:22.964Z)

I?ve got the thread state back. I?m going to do the last bit of repo hygiene now: confirm we?re still on the PR branch, stage only the smoke-test CLI/doc/test files, commit, push, and then I?ll give you the exact WAI verification results.

113. assistant (2026-07-09T03:40:29.753Z)

We?re on the right branch and HEAD is the PR work, with only the expected four tracked files changed plus the local untracked manifest from the earlier real migration. I?m staging only the CLI/docs/tests files so that migration metadata does not sneak into this code PR.

114. assistant (2026-07-09T03:40:38.312Z)

The focused suite and lint are green after staging. I?m going to run the new smoke command once more against the CLI itself, then commit and push it onto the review PR.

115. assistant (2026-07-09T03:40:48.732Z)

The smoke command itself passes and exercises the path we care about, but invoking it as `python -m storagectl.storagectl` exposes a Python module warning from package import order. That?s not a functional failure, but it?s exactly the sort of paper cut a smoke mode should avoid, so I?m going to clean that up before pushing.

116. assistant (2026-07-09T03:40:56.225Z)

The warning comes from `storagectl/__init__.py` eagerly importing `storagectl.storagectl`, then `python -m storagectl.storagectl` tries to execute the same module. I?m switching that package export to lazy loading so both console-script use and module use stay quiet.

117. assistant (2026-07-09T03:41:08.925Z)

That import cleanup is in. I?m rerunning the same checks now, including the module invocation, because a smoke command should be calm when someone copies the documented fallback command.

118. assistant (2026-07-09T03:41:19.827Z)

Clean now: tests pass, ruff passes, and the smoke command exits with `# smoke-test passed` without the module warning. I?m doing the final branch/HEAD check and pushing this onto PR #5.

119. assistant (2026-07-09T03:41:30.534Z)

The smoke-test commit is created. One parallel log check raced just before the commit landed, so I?m rechecking the PR branch commit list now before the push.

120. assistant (2026-07-09T03:41:37.907Z)

The PR branch now contains only the three intended commits, and the only leftover working tree item is the untracked local manifest from the earlier real one-file migration. Pushing the PR update now.

121. assistant (2026-07-09T03:41:51.443Z)

The branch is pushed to PR #5 with the new smoke command. I?m doing one quick handoff check because this repo uses a living session note, and a pushed CLI validation mode is worth recording for the next session.

122. assistant (2026-07-09T03:41:57.000Z)

The handoff is stale around PR #4 and doesn?t mention the smoke mode yet. I?m refreshing just that session note, then I?ll make one small follow-up commit so the PR carries accurate resume context too.

123. assistant (2026-07-09T03:42:15.817Z)

The handoff now reflects PR #5?s actual scope and the smoke-test verification. I?m committing that single doc refresh and pushing it to the same PR.

124. assistant (2026-07-09T03:42:21.464Z)

Still on the PR branch, and only the handoff plus the intentionally untracked local manifest are present. Staging just the handoff now.

125. assistant (2026-07-09T03:42:28.012Z)

The staged file name is correct, but the first stat view was blank, likely because this is mostly a small line-edit diff with Windows line-ending noise in play. I?m checking the actual staged patch before committing.

126. assistant (2026-07-09T03:42:33.115Z)

The staged handoff patch is exactly the resume-context update, with no manifest staged. Committing and pushing that now.

127. assistant (2026-07-09T03:42:40.036Z)

That log check raced with the commit again, same harmless timing issue. I?ll do the final post-commit log and push.

128. assistant (2026-07-09T03:42:51.846Z)

Yes. It did not have a dedicated smoke-test mode before, so I added one to PR #5:

[PR #5: Add vault-driven batch population command](https://github.com/Khelsutra/rally-corpus-vault/pull/5)

New command:

```powershell
storagectl smoke-test
```

What it verifies:
- creates a temporary mini-vault
- runs `batch-add` into temporary F/G tiers
- verifies both tiers
- reruns the batch to prove idempotence
- checks progress-log markers like `BATCH_START`, `PROGRESS copy ...`, and `BATCH_SUCCESS`

I used it locally and confirmed WAI:

```text
python -m pytest tests                          -> 35 passed
python -m ruff check .                          -> clean
python -m storagectl.storagectl smoke-test ... -> # smoke-test passed
```

I also fixed a small `python -m storagectl.storagectl` module warning so the fallback invocation is clean.

PR #5 is pushed and ready for review. The only uncommitted local item left is the untracked `manifests/` from the earlier real one-video migration smoke, which I intentionally did not include in the code PR.

129. user (2026-07-09T04:03:01.321Z)

pls review comments on the PR

130. assistant (2026-07-09T04:03:09.700Z)

I?ll pull the PR branch first per your repo convention, then I?ll inspect the review/comment threads and separate ?must fix? items from discussion.

131. assistant (2026-07-09T04:03:19.439Z)

I?m using the GitHub PR comment handler workflow here so I can preserve thread state, not just flat comment text. The local tree only has the untracked migration manifest, so a fast-forward pull should be safe.

132. assistant (2026-07-09T04:03:28.598Z)

The PR has one review with two required bug fixes called out. I?m fetching the inline review threads now so I can patch against the exact file/line comments rather than guessing from the review summary.

133. assistant (2026-07-09T04:03:46.398Z)

I found five unresolved threads: two are marked bugs and block merge; three are suggestions/nit-level. I?m going to address the two required bugs now, and I?ll also take the tiny low-risk logging-label suggestion if it falls out cleanly.

134. assistant (2026-07-09T04:03:55.445Z)

The review summary was accurate: the current merge path treats `False` and a default lifecycle as authoritative, and the existing-copy verify path hashes without progress notes. I?ve got the relevant code and tests open; next I?m patching those two behaviors with regressions.

135. assistant (2026-07-09T04:04:19.872Z)

I?m making the merge semantics explicit: `--golden` can promote a record, but absence of

`--golden` will no longer demote it; lifecycle defaults fill blanks but won?t erase ratification/GCS state. For the liveness bug, existing-copy verification will use the same progress logger as copy verification.

136. assistant (2026-07-09T04:04:42.390Z)

The core patch is in, including the small `BATCH_START` improvement so the log now reports source volume and planned write volume. I?m adding focused regression tests next, including one that proves resume/idempotent re-run emits progress while re-hashing existing tier copies.

137. assistant (2026-07-09T04:05:09.776Z)

The regression tests are added. I?m running the suite and lint now; if anything shakes loose, I?ll tighten the patch before touching the PR branch.

138. assistant (2026-07-09T04:05:33.478Z)

Checks are green with 37 tests now. I?m going to commit the fixes for the two blocking review bugs plus the low-risk batch volume log improvement, leaving the temp-file and per-tier exception suggestions as follow-up discussion unless you want those folded into this PR too.

139. assistant (2026-07-09T04:05:48.774Z)

The diff is scoped to the reviewed code paths and tests; the local untracked manifest is still untouched. I?m staging only these two files, committing, and pushing to the PR.

140. assistant (2026-07-09T04:06:04.037Z)

The staged file list is correct. I?m doing the final branch/HEAD check before committing, following the shared-checkout safety rule.

141. assistant (2026-07-09T04:06:15.746Z)

The safety check is clean: right branch, expected HEAD, and only the two intended files staged. Committing now.

142. assistant (2026-07-09T04:06:23.439Z)

The commit landed. As before, the parallel log check missed the just-created commit by a moment, so I?m rechecking the branch delta and then pushing.

143. assistant (2026-07-09T04:07:00.239Z)

The push succeeded. Two read-only status commands timed out around the same time, so I?m doing a quick clean confirmation now and then I?ll report what was addressed.

144. assistant (2026-07-09T04:07:16.886Z)

Reviewed the PR comments and pushed fixes to PR #5:

https://github.com/Khelsutra/rally-corpus-vault/pull/5

Addressed:
- Fixed golden/lifecycle clobber on incremental `add` / `batch-add` reruns.
- Fixed silent resume behavior: existing-copy verification now emits progress logs while re-hashing F/G copies.
- Also applied the small `BATCH_START` logging suggestion: it now reports `source_gb` and `planned_write_gb`.

Added regressions:
- incremental second-tier add preserves `golden=True` and existing lifecycle ratification/GCS state

- batch rerun logs `PROGRESS verify existing ...`
- batch log includes source/write volume fields

Verified:
```text
python -m pytest tests                          -> 37 passed
python -m ruff check .                          -> clean
python -m storagectl.storagectl smoke-test ... -> # smoke-test passed
```

I did not resolve or reply to GitHub threads because you only asked me to review comments, not perform GitHub review-thread writes. The remaining unresolved comments are suggestions around temp-file cleanup/capacity edge cases and controlled per-tier I/O exception handling.

145. user (2026-07-09T04:07:54.743Z)

pls respond to the threads as well so the review process can continue

146. assistant (2026-07-09T04:08:18.142Z)

I?ll add concise replies to each unresolved review thread, pointing to the pushed commit and calling out what was fixed versus what remains intentionally follow-up. I won?t resolve the threads unless you explicitly want me to mark them resolved.

147. assistant (2026-07-09T04:08:35.054Z)

I?ve got the live thread IDs. Two comments will get ?fixed in commit + test coverage? replies, one suggestion got partially addressed, and the two remaining suggestions will be acknowledged as follow-up hardening rather than blocking the migration command.

148. assistant (2026-07-09T04:09:11.181Z)

First blocking bug thread has a reply with the commit and regression test. I?m replying to the resume-progress bug next.

149. assistant (2026-07-09T04:09:20.006Z)

Both required bug threads now have direct responses. I?ll add the three suggestion replies so the reviewer has a clear disposition for each.

150. assistant (2026-07-09T04:09:28.092Z)

Temp-file/capacity suggestion is acknowledged as follow-up. I?m replying to the per-tier I/O exception handling suggestion now.

151. assistant (2026-07-09T04:09:37.111Z)

The exception-handling suggestion has a clear follow-up disposition. Last one is the batch volume logging thread, which was addressed in the pushed commit.

152. assistant (2026-07-09T04:09:45.318Z)

All five review threads have replies now. I?ll do a quick thread-state readback so I can tell you exactly what remains open.

153. assistant (2026-07-09T04:09:53.283Z)

Done. I replied to all five PR review threads.

Summary:
- Marked the two bug threads as fixed in `3a9a7b0`, with the added regression tests and verification commands.
- Marked the `BATCH_START` volume logging suggestion as addressed.

- Acknowledged the temp-file/capacity and per-tier I/O exception suggestions as follow-up hardening, not part of the current blocking fix.

I left the threads unresolved, since you asked for replies rather than thread resolution.

154. user (2026-07-09T04:09:53.704Z)

please mark as resolved the ones which you have actually resolved

155. assistant (2026-07-09T04:09:58.154Z)

I?ll resolve only the threads that are actually fixed in the pushed code: golden/lifecycle preservation, existing-copy progress logging, and batch volume logging. I?ll leave the two follow-up hardening suggestions open.

156. assistant (2026-07-09T04:10:05.919Z)

Those three are now marked resolved. I?m doing a quick readback to make sure only the two follow-up suggestion threads remain open.

157. assistant (2026-07-09T04:10:13.908Z)

Done. Resolved the three threads that were actually fixed:

- golden/lifecycle clobber
- existing-copy resume progress logging
- `BATCH_START` source/write volume logging

Left open:
- stale temp-file / replacement capacity hardening
- controlled per-tier I/O exception handling

Those two are still valid follow-up suggestions, not fixed in the current PR.

158. user (2026-07-09T04:15:27.301Z)

can you take another pass and merge after responding to the comments and ensuring the follow ups are taken up

159. assistant (2026-07-09T04:15:40.370Z)

I?ll take one more pass end-to-end: refresh the branch, inspect the current PR thread state, implement the two follow-up hardening items if they still stand, reply/resolve the threads that are actually fixed, run the local gates, then merge the PR.

160. assistant (2026-07-09T04:15:48.074Z)

The only local leftover is still the untracked migration manifest; I?ll keep that out of the PR. Now I?m doing the required fast-forward pull and reading the live PR thread state before editing.

161. assistant (2026-07-09T04:15:53.158Z)

The PR is clean to merge and CI is currently green, but two review suggestions are still open. I?m going to implement both in this PR so the unattended migration path is stronger before we merge.

162. assistant (2026-07-09T04:16:03.698Z)

There are new review comments from the second pass, and a couple are real merge-contract bugs rather than cosmetic feedback. I?m going to fold them in now: metadata merge preservation, log-write resilience, controlled per-row exception handling, temp orphan cleanup/diagnostics, and the missing tests.

163. assistant (2026-07-09T04:16:54.068Z)

I?ve got the patch shape: make `upsert_video` merge dictionaries instead of replacing them, make progress-log mirroring non-fatal, convert per-target copy/hash failures into controlled row failures, clean stale temp files for the target object, and add `add --log/--progress-interval` so single-video re-verification has the same liveness path.

164. assistant (2026-07-09T04:17:06.987Z)

The merge/logging part is in. I?m moving to the filesystem and batch-control parts now: temp cleanup, digest diagnostics, and row-level failure handling that still records successful tier copies.

165. assistant (2026-07-09T04:17:30.816Z)

Per-target failures now turn into controlled row failures while preserving any successful tier entries. Next I?m wrapping the batch row call so `--continue-on-error` also applies to unexpected exceptions, then I?ll expose progress flags on single-file `add`.

166. assistant (2026-07-09T04:18:04.218Z)

The batch exception path now logs exactly one row failure per unexpected exception. I?m adding the small test fixtures now so the merge contract and unattended failure behavior are pinned down.

167. assistant (2026-07-09T04:19:05.857Z)

The main missing regression cases are in place. I noticed a small cleanup needed in the test helper itself, so I?m tightening that before running the suite.

168. assistant (2026-07-09T04:19:18.743Z)

I?m also checking the runbook wording that the review called out, so the docs match the behavior: existing copies are re-verified, not magically skipped, and crash cleanup is documented.

169. assistant (2026-07-09T04:19:28.585Z)

The new tests found exactly one expectation mismatch: the merge code correctly fills an empty `f_ingest_path`; the risk is replacing an existing one. I?m adjusting that test to cover the real clobber case, then I?ll update the runbook wording from ?skipped? to ?re-verified without recopying.?

170. assistant (2026-07-09T04:19:45.391Z)

The first runbook patch missed the exact recovery wording, so I?m reading that small section and applying the doc update against the actual text.

171. assistant (2026-07-09T04:19:48.554Z)

I found the exact runbook language. Applying the doc correction now.

172. assistant (2026-07-09T04:19:59.196Z)

The docs now match the behavior. I?m rerunning the gates and the CLI smoke test; this pass includes the new failure-mode tests.

173. assistant (2026-07-09T04:20:13.760Z)

All local gates are green with 42 tests now. I?m checking the repo?s CI/test config and the final diff before I commit, so the merge has the same confidence as the local run.

174. assistant (2026-07-09T04:20:22.976Z)

The diff is scoped to the review threads, and CI config is just pytest plus ruff. Since we?re about to merge, I?m also running coverage locally even though the repo doesn?t define a fail-under threshold.

175. assistant (2026-07-09T04:20:35.617Z)

Coverage is green as well, with no repo fail-under configured. I?m checking for generated coverage artifacts and then I?ll stage only the review-fix files.

176. assistant (2026-07-09T04:20:43.119Z)

The coverage run created a local `.coverage` artifact, so I?m removing that generated file and keeping the migration manifest untracked. Then I?ll commit just the three review-fix files.

177. assistant (2026-07-09T04:20:47.723Z)

The cleanup/status raced with staging and still shows `.coverage`, so I?m doing the artifact cleanup once more serially and confirming before committing.

178. assistant (2026-07-09T04:20:56.204Z)

`.coverage` is being stubborn, likely because the coverage process left a file handle or it was recreated after the parallel check. I?m inspecting it briefly and removing it with a full path.

179. assistant (2026-07-09T04:21:00.248Z)

No Python process is holding it. I?m removing the exact absolute path now.

180. assistant (2026-07-09T04:21:09.556Z)

Generated coverage is gone; only the three staged review-fix files and the intentionally untracked manifest remain. I?m doing the branch/HEAD check and committing.

181. assistant (2026-07-09T04:21:16.102Z)

We?re on the PR branch with the expected staged patch. Committing and pushing now.

182. assistant (2026-07-09T04:21:22.629Z)

The commit landed. I?ll re-read the branch delta after the commit, push, then respond to the new PR threads with the exact fixes.

183. assistant (2026-07-09T04:21:35.715Z)

The follow-up hardening commit is pushed. I?m replying to and resolving the threads that are now actually fixed, including the older open suggestion threads that the new commit closes.

184. assistant (2026-07-09T04:22:00.658Z)

That reply loop used PowerShell hashtable property syntax incorrectly, so GitHub rejected every mutation and nothing was posted. I?m rerunning it with the correct key access.

185. assistant (2026-07-09T04:22:39.815Z)

PowerShell is still passing the hashtable accessor text through to `gh` instead of evaluating it in-place. I?m going to avoid the loop and post the replies one-by-one with literal thread IDs; less elegant, more reliable.

186. assistant (2026-07-09T04:22:59.552Z)

The first four thread replies posted successfully. I?m posting the remaining four now, then I?ll resolve all eight fixed threads.

187. assistant (2026-07-09T04:23:03.879Z)

All replies are posted. I?m resolving those eight now because they correspond to code/docs/tests fixed in `58eff28`.

188. assistant (2026-07-09T04:23:25.474Z)

The fixed threads are resolved. I?m reading back the PR state now: unresolved threads, CI rollup, and mergeability before the merge command.

189. assistant (2026-07-09T04:23:41.185Z)

Everything is resolved and CI is green. Before merging, I?m refreshing the handoff so `master` won?t say ?review/merge PR #5? immediately after PR #5 is merged.

190. assistant (2026-07-09T04:23:45.116Z)

The handoff still has the old pending-review language. I?m changing it to the actual next phase: run smoke and execute the vault-driven migration.

191. assistant (2026-07-09T04:24:02.578Z)

Handoff is refreshed with the current verification count and migration next step. I?ll commit this doc-only update, push it, and confirm CI after the final head.

192. assistant (2026-07-09T04:24:10.282Z)

Only the handoff is staged, on the correct branch. Committing and pushing the final doc refresh now.

193. assistant (2026-07-09T04:24:18.919Z)

The commit succeeded; the status read raced just before it completed. I?m rechecking and pushing the final head.

194. assistant (2026-07-09T04:24:27.244Z)

Final head is pushed. I?m waiting for the PR check on this final commit before merging.

195. assistant (2026-07-09T04:24:35.361Z)

CI is queued on the final handoff-only commit, so I?m going to poll until it completes rather than merging on an unstable head.

196. assistant (2026-07-09T04:25:07.124Z)

CI is green on the final commit and merge state is clean. I?m checking the repo?s recent merge style, then I?ll merge PR #5 using the same convention if it?s clear.

197. assistant (2026-07-09T04:25:13.352Z)

Recent history uses squash-style PR merge commits (`Document vault onboarding and migration plan (#4)`), so I?ll squash-merge this one too and delete the remote PR branch.

198. assistant (2026-07-09T04:25:32.661Z)

PR #5 is merged. I?m doing the post-merge sanity check now: local branch, master head, PR state, and the only remaining local untracked migration manifest.

199. assistant (2026-07-09T04:25:40.077Z)

Local checkout is now `master` at the merge commit. I?ll run the CLI smoke test once more from merged `master` so the next migration step starts from the exact code that landed.

200. assistant (2026-07-09T04:25:53.617Z)

Post-merge smoke and tests pass on `master`. I?m doing one final status check so I can hand you a clean next step for the real migration.

201. assistant (2026-07-09T04:26:07.543Z)

Done. PR #5 is merged into `master`:

https://github.com/Khelsutra/rally-corpus-vault/pull/5
Merge commit: `40d08f9`

I took the follow-ups into the PR before merging:
- preserved `source_capture` and `full` metadata on sparse re-adds
- made progress-log write failures non-fatal
- made per-tier and per-row failures controlled/resumable
- cleaned stale `*.tmp-*` target temp files and added hash-mismatch diagnostics
- added `add --log` / `--progress-interval`
- refreshed runbook + handoff
- replied to and resolved the fixed review threads

Verified:
```text
CI on final PR head -> success
python -m pytest tests -> 42 passed
python -m ruff check . -> clean
python -m pytest --cov=storagectl tests -> 42 passed, 72% coverage
post-merge smoke-test on master -> passed
```

Local checkout is now `master` at `40d08f9`. The only remaining local untracked item is `manifests/`, from the earlier one-video real migration smoke, intentionally not committed.

202. developer (2026-07-09T04:26:44.752Z)

<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>

203. user (2026-07-09T04:26:44.780Z)

please proceed with the actual vault migration now once the real smoke tests pass and do make sure a mummary is given at the end by the vault tool

204. assistant (2026-07-09T04:27:08.167Z)

I?ll proceed from merged `master`: pull first, read the handoff/runbook, run the real local smoke gates, then let `storagectl batch-add` drive the migration with the progress log. At the

end I?ll run the vault?s own `status`/`verify` commands so the summary comes from the tool, not just from my narration.

205. assistant (2026-07-09T04:27:20.248Z)

The checkout is current on `master` at the merged PR. The only untracked item is still the local `manifests/` from the earlier one-video smoke, which is expected for the real migration state. I?m reading the handoff/runbook and checking the local vault config before starting any data movement.

206. assistant (2026-07-09T04:27:43.774Z)

The runbook says the real preflight is: tool smoke test, tests/lint, external tool availability, drive free space, source presence, and current vault status/verify. I?m running those now; if any source or tier check fails, I?ll stop before moving data.

207. assistant (2026-07-09T04:28:00.018Z)

The code/test smoke gates are green, `ffprobe` and `gcloud` are available, and both drives have enough free space for this batch. I?m checking the 15 real source files and the current one-video vault state now.

208. assistant (2026-07-09T04:28:15.040Z)

The real smoke gates passed: all 15 sources are present, total source size is 111.87 GB, and the existing vault has 1 verified golden video on both F and G with zero N+1 violations. I?m starting the vault-driven batch now; the only copy operation will be `storagectl batch-add`, with its own 30-second progress log and per-row manifest persistence.

209. assistant (2026-07-09T04:28:40.816Z)

The batch process was launched as a hidden native Windows process so the vault command can run uninterrupted. I?m checking its PID and the progress log now.

210. assistant (2026-07-09T04:28:48.342Z)

The vault batch is running. It has started `adarsh_avi_singles`; the progress log shows `BATCH_START` for 15 videos with `source_gb=120.12` and `planned_write_gb=240.24`, and it is currently hashing the first 11.45 GB source.

211. assistant (2026-07-09T04:29:27.259Z)

The vault has finished hashing `adarsh_avi_singles`, copied the F-tier canonical file, and is re-hash-verifying that F copy now. No stderr so far.

212. assistant (2026-07-09T04:30:05.308Z)

`adarsh_avi_singles` has copied and verified on F, copied to G, and is verifying the G copy. The vault is still clean: no stderr and progress is being written as designed.

213. assistant (2026-07-09T04:30:46.363Z)

First row completed: `adarsh_avi_singles` is recorded on both F and G and the manifests were written to repo/F/G. The vault is now on item 2/15, `kushagra_singles`, copying its F-tier canonical file after a completed source hash.

214. assistant (2026-07-09T04:31:25.784Z)

`kushagra_singles` is copied and verified on F; the vault is now copying it to G. Still no stderr, and the row-level progress log is updating normally.

215. assistant (2026-07-09T04:32:04.329Z)

Second row completed and manifests were written again to all three locations. The vault is now on item 3/15, `largetest_doubles`, hashing an 11.53 GB source.

216. assistant (2026-07-09T04:32:41.441Z)

`largetest_doubles` finished source hashing and is copying to F. The vault remains healthy with no stderr.

217. assistant (2026-07-09T04:33:18.957Z)

`largetest_doubles` is verified on F and is now copying to G. Progress is still moving at full-file scale with no stderr.

218. assistant (2026-07-09T04:33:57.292Z)

`largetest_doubles` has copied to G and is now verifying the G copy. Still no errors from the vault process.

219. assistant (2026-07-09T04:34:34.867Z)

Third row completed and manifests were written. The vault is now on item 4/15, `gbaaddy`; it has finished source hashing and started the F copy.

220. assistant (2026-07-09T04:35:12.686Z)

`gbaaddy` is copied and verified on F; the vault is copying it to G now. No stderr.

221. assistant (2026-07-09T04:35:55.351Z)

Fourth row completed and was recorded. The vault is on item 5/15, `testlarge_short`; its source hash and F copy are already done, and it is verifying F.

222. assistant (2026-07-09T04:36:32.233Z)

Rows 5 and 6 are complete. `mahadevpura_singles` was correctly treated as already present and re-verified on both tiers without recopying; the vault is now on item 7/15, `mahadevpura_2`, copying to F.

223. assistant (2026-07-09T04:37:14.899Z)

`mahadevpura_2` is now copied and verified on F; G copy has started. No errors.

224. assistant (2026-07-09T04:37:51.445Z)

`mahadevpura_2` finished G copy and is now verifying G. It is moving steadily; no stderr from the vault.

225. assistant (2026-07-09T04:38:30.515Z)

Row 7 completed and manifests were persisted. The vault is on item 8/15, `GX010128`, and has started copying to F after hashing the 11.33 GB source.

226. assistant (2026-07-09T04:39:15.035Z)

`GX010128` is copied and verified on F; G copy has started. The vault process is still clean.

227. assistant (2026-07-09T04:39:54.090Z)

`GX010128` has finished the G copy and is verifying on G. Still no stderr.

228. assistant (2026-07-09T04:40:30.798Z)

Row 8 completed and was recorded. The vault is on item 9/15, `mahadevpura_1`; the small F copy

is done and F verification is underway.

229. assistant (2026-07-09T04:41:07.787Z)

Row 9 completed. The vault is on item 10/15, `GX030094`, hashing a 10.61 GB source. No errors.

230. assistant (2026-07-09T04:41:44.541Z)

`GX030094` has finished hashing and the F copy; the vault is verifying F now. No stderr.

231. assistant (2026-07-09T04:42:21.558Z)

`GX030094` is verified on F and is copying to G; last progress shows about 86% of the G copy. No errors.

232. assistant (2026-07-09T04:42:58.253Z)

`GX030094` has completed the G copy and is verifying G. Still no stderr.

233. assistant (2026-07-09T04:43:36.031Z)

Row 10 completed and was recorded. The vault is on item 11/15, `Badminton_BXH_2`, copying to F after hashing 10.93 GB.

234. assistant (2026-07-09T04:44:12.982Z)

`Badminton_BXH_2` is copied and verified on F; G copy has started. No stderr.

235. assistant (2026-07-09T04:44:52.325Z)

`Badminton_BXH_2` has copied to G and is verifying on G. Still clean.

236. assistant (2026-07-09T04:45:30.011Z)

Row 11 completed and manifests were persisted. The vault is on item 12/15, `GX010141`, hashing 5.77 GB.

237. assistant (2026-07-09T04:46:10.995Z)

`GX010141` is copied and verified on F; it is now copying to G. Still no stderr.

238. assistant (2026-07-09T04:46:49.218Z)

Row 12 completed and was recorded. The vault is on item 13/15, `GX010137`, and the source hash is already done. Still no stderr.

239. assistant (2026-07-09T04:47:27.769Z)

`GX010137` is copied and verified on F; it is copying to G now. No stderr.

240. assistant (2026-07-09T04:48:05.398Z)

Row 13 completed and was recorded. The vault is on item 14/15, `Boxhill_Doubles`, hashing an 11.53 GB source. No stderr.

241. assistant (2026-07-09T04:48:44.626Z)

`Boxhill_Doubles` finished source hashing and is copying to F. There are a few short-lived Python children visible, but the main vault process is still the original one and stderr remains empty.

242. assistant (2026-07-09T04:49:24.518Z)

`Boxhill_Doubles` is copied and verified on F; the G copy has started. No vault errors.

243. assistant (2026-07-09T04:50:02.079Z)

`Boxhill_Doubles` has finished copying to G and is verifying the G copy. No stderr.

244. assistant (2026-07-09T04:50:39.342Z)

Row 14 completed and manifests were written. The vault is on the final item, `GX020094`, hashing 11.53 GB. Almost there.

245. assistant (2026-07-09T04:51:19.513Z)

Final item `GX020094` has finished hashing and copying to F; it is verifying F now. After this comes the final G copy and verify.

246. assistant (2026-07-09T04:51:58.440Z)

Final item is verified on F and now copying to G. No stderr; after this, one final G verify and the batch should close with `BATCH_SUCCESS`.

247. assistant (2026-07-09T04:52:35.933Z)

The final G copy is done and the vault is in the last verification pass for `GX020094`. One more poll should catch the batch close and then I?ll run the vault?s final status/verify summary.

248. assistant (2026-07-09T04:53:21.779Z)

The batch closed cleanly with `BATCH_SUCCESS`; all 15 rows completed and the vault wrote the manifest to the repo, F, and G. I?m moving into the vault?s final summary gates now: `status`, then full `verify F`, then full `verify G`.

249. assistant (2026-07-09T04:53:30.747Z)

Vault status summary is exactly what we wanted: 15 videos, F/G each at 120.1 GB full-res, zero N+1 violations. I?m running the full F-tier hash verification now.

250. assistant (2026-07-09T04:53:46.619Z)

`verify F` is running as a vault process. This command does full-file MD5 reads for all 15 F-tier masters, so it can take a little while even though no copy is happening.

251. assistant (2026-07-09T04:54:26.151Z)

Tiny monitor hiccup: I accidentally used PowerShell?s reserved `$PID` variable while polling, so that status line is not trustworthy. The verify process itself is still separate; I?m rechecking with a non-reserved variable name.

252. assistant (2026-07-09T04:54:34.395Z)

`verify F` is still running and has not emitted any errors. I?ll keep polling it until the vault gives the summary line.

253. assistant (2026-07-09T04:55:12.302Z)

`verify F` is still hashing through the F-tier masters. No stderr; just waiting for the final vault summary.

254. assistant (2026-07-09T04:55:51.658Z)

`verify F` is still running cleanly. These full-read verifications are deliberately boring, but they are the proof we need.

255. assistant (2026-07-09T04:56:33.784Z)

`verify F` is still in progress with no stderr. I?m continuing to poll; once it returns, I?ll immediately start `verify G`.

256. assistant (2026-07-09T04:57:11.788Z)

`verify F` passed: `15 ok, 0 missing, 0 hash-mismatch`. I?m starting the G-tier verification now, which is the final full-byte proof for the migration.

257. assistant (2026-07-09T04:57:26.500Z)

`verify G` is now running. It will read the 15 G-tier masters and report the vault summary when it finishes.

258. assistant (2026-07-09T04:58:04.186Z)

`verify G` is still running with no stderr. Continuing the final verification pass.

259. assistant (2026-07-09T04:58:41.731Z)

`verify G` is still hashing through the G copies. No errors reported.

260. assistant (2026-07-09T04:59:20.197Z)

`verify G` is still running cleanly. G reads are slower than F, but the process is alive and quiet.

261. assistant (2026-07-09T04:59:58.059Z)

`verify G` is still in progress and still has no stderr. I?m continuing to wait for the vault?s final line.

262. assistant (2026-07-09T05:00:36.514Z)

`verify G` is still running. No vault errors; just full-file verification taking its time on G.

263. assistant (2026-07-09T05:01:15.439Z)

`verify G` is still in the final hash pass with no stderr. Staying with it.

264. assistant (2026-07-09T05:01:53.185Z)

`verify G` is still running cleanly. The batch itself is already complete; this is the post-migration full verification.

265. assistant (2026-07-09T05:02:31.337Z)

`verify G` is still active and quiet. Continuing; I won?t move on until the vault reports the G result.

266. assistant (2026-07-09T05:03:12.450Z)

`verify G` passed too: `15 ok, 0 missing, 0 hash-mismatch`. I?m doing the manifest identity checks now, then I?ll commit only the portable manifest metadata.

267. assistant (2026-07-09T05:04:14.151Z)

I?m picking up from the completed migration state. I?ll do the last bit of discipline now: verify the repo state, commit only the generated vault metadata, refresh the handoff, and push so the vault?s view is portable for the next machine.

268. assistant (2026-07-09T05:04:24.485Z)

The checkout is still at remote head and the migration artifacts are local. I?m checking the branch, latest commit, and the exact file set before staging anything.

269. assistant (2026-07-09T05:04:31.318Z)

Only the repository manifest is untracked, which is what we expect after the vault tool writes portable metadata. I?m giving it one quick sanity read before committing it.

270. assistant (2026-07-09T05:04:38.699Z)

The manifest contains 15 portable vault entries plus the original ingest-path provenance for traceability. I?m staging only `manifests/corpus_index.json` now, no scratch logs, local config, or media.

271. assistant (2026-07-09T05:04:46.623Z)

Staged path is intentionally narrow. I?ll verify the staged set before creating the metadata commit.

272. assistant (2026-07-09T05:04:50.251Z)

Everything staged belongs to the vault metadata commit, and HEAD is still the merged command PR. Committing that manifest now.

273. assistant (2026-07-09T05:05:02.052Z)

The manifest commit is created locally. I?m refreshing the session handoff next so future sessions start from the actual migrated state, then I?ll push both commits in order.

274. assistant (2026-07-09T05:05:10.300Z)

The handoff is stale: it still points at ?PR #5 ready? and pre-migration next steps. I?m going to push the manifest commit first, then rewrite the handoff against the shipped state instead of recording a half-step.

275. assistant (2026-07-09T05:05:17.894Z)

The portable corpus manifest is now on `origin/master` as `af6f6da`. I?m updating the handoff so the next session sees ?migration complete? immediately, with the exact verification results and where to go next.

276. assistant (2026-07-09T05:05:42.986Z)

The handoff now matches the completed migration. I?ll commit that single docs file separately so the manifest commit stays clean.

277. assistant (2026-07-09T05:05:46.140Z)

Branch and HEAD are exactly where expected; only the handoff changed. Staging just that file.

278. assistant (2026-07-09T05:05:55.930Z)

The staged set is only the handoff. Committing it now, then I?ll push and run one last vault status check from the committed state.

279. assistant (2026-07-09T05:06:05.096Z)

The handoff commit is local. I?ll push it and then do the final vault-tool summary check the user asked for.

280. assistant (2026-07-09T05:06:08.150Z)

Only the handoff commit is ahead of origin now, and the working tree is clean. Pushing it.

281. assistant (2026-07-09T05:06:21.353Z)

The commits are on `origin/master`. I?m running the vault?s final status summary now, plus a clean-tree check so we close with both data and repo state confirmed.

282. assistant (2026-07-09T05:06:42.102Z)

Done. The actual vault migration completed, and the vault tool?s final summary is clean:

```text
# corpus: 15 unique videos across tiers ['F', 'G', 'GCS']
  F: 120.1 GB full-res
  G: 120.1 GB full-res
  GCS: 0.0 GB full-res

N+1 violations (golden/ratified with <2 durable verified copies): 0
```

Pre-migration gates passed:
- `python -m pytest tests` -> 42 passed
- `python -m ruff check .` -> clean
- `python -m storagectl.storagectl smoke-test --log scratch/pre_migration_smoke.log` -> passed

Migration command completed with `BATCH_SUCCESS`:
- `python -m storagectl.storagectl batch-add seed/golden_bootstrap_plan.windows.csv --to F,G --log scratch/bootstrap_golden.log --progress-interval 30`

Post-migration verification passed:
- `storagectl verify F` -> 15 ok, 0 missing, 0 hash-mismatch
- `storagectl verify G` -> 15 ok, 0 missing, 0 hash-mismatch
- Repo/F/G manifest SHA256 all match: `379399BCAFAC60359DE91ED72D50B38C010CB602D9BF05C06CDDDA1EE733C90E`

I committed and pushed the portable metadata plus updated handoff:
- `af6f6da` `Bootstrap golden corpus manifest`
- `5fe9bb0` `Update handoff after golden migration`

Current repo state is clean, and `origin/master` is at `5fe9bb0`. The vault now has the 15-video golden corpus on both F: and G: with N+1 satisfied.

283. user (2026-07-09T05:06:42.683Z)

we should have a %age or some more finer gained logging to understand what is happening in these long runs

284. assistant (2026-07-09T05:06:52.414Z)

Agreed. The 30-second heartbeat proves the process is alive, but for 100+ GB copies it should also say how far through the current file and batch we are. I?ll add byte/percent/rate/ETA style progress where the vault already emits long-run notes.

285. assistant (2026-07-09T05:07:02.133Z)

I?m going to branch from `origin/master` explicitly, then trace the batch/add copy path so the new logging fits the existing CLI instead of being a one-off print.

286. assistant (2026-07-09T05:07:09.151Z)

The existing logger is already centralized in `ProgressLogger.progress`, so this should be a compact change: improve the one formatter and add batch-level context around each file. That?s a nice little pressure point.

287. assistant (2026-07-09T05:07:21.722Z)

Branch is cut from `origin/master`. I?m reading the logger and tests around batch-add now so the smoke mode also proves the new format is working.

288. assistant (2026-07-09T05:07:39.628Z)

One interesting find: the logger already prints per-operation percent, but not rate/ETA or whole-batch percent, so the run can still feel opaque. I?m adding both: richer phase lines and explicit batch item progress before each video.

289. assistant (2026-07-09T05:07:53.654Z)

I?m going to keep the log human-readable and grep-friendly: existing markers stay intact, and each progress line gets `rate_mb_s`, `elapsed_s`, and `eta_s`; batch item start/done lines get item size and overall batch percentages.

290. assistant (2026-07-09T05:08:07.533Z)

I?m making the edit now in the CLI and its tests. The output markers that scripts may already depend on will remain prefixes, with extra fields appended.

291. assistant (2026-07-09T05:08:23.759Z)

The phase-level progress lines are richer now. I?m wiring overall batch percentages into the batch loop so the log can answer both ?what is this file doing?? and ?where are we in the whole run??.

292. assistant (2026-07-09T05:08:46.646Z)

The core behavior is patched. I?m tightening the tests so they check for the new fields, not just the old ?it logged something? marker.

293. assistant (2026-07-09T05:09:08.976Z)

Tests now protect the richer output. I?m also updating the runbook language so future operators know what to expect in long logs.

294. assistant (2026-07-09T05:09:19.084Z)

The implementation is small enough that I?ll run the focused logging tests first, then the full suite and smoke-test. If anything gets weird, it should show up quickly in the batch-add path.

295. assistant (2026-07-09T05:09:40.686Z)

Focused tests pass. I noticed one small readability wrinkle in the failure branch while reviewing the diff, so I?m cleaning that up before the full verification run.

296. assistant (2026-07-09T05:09:48.299Z)

That cleanup is in. Now I?m running the real checks: full test suite, lint, and the CLI smoke test that exercises batch-add end to end.

297. assistant (2026-07-09T05:10:04.693Z)

The checks passed. The smoke output exposed a cosmetic issue on tiny files: the first progress line can show `rate_mb_s=0.0` because timing starts at the first progress event. I?m shifting the timer to phase start so real long-run rates/ETA are less misleading.

298. assistant (2026-07-09T05:10:19.976Z)

Timer start is corrected. I?m rerunning the full verification set after that final tweak.

299. assistant (2026-07-09T05:10:33.874Z)

All checks are green again, including the real `storagectl smoke-test`. I?m checking the final diff and repo state now, then I?ll commit this on the feature branch.

300. assistant (2026-07-09T05:10:44.647Z)

The diff is scoped to the logger, its tests, and the runbook. I?m doing the pre-commit branch/HEAD check, then I?ll stage only those three files.

301. assistant (2026-07-09T05:10:48.213Z)

HEAD is exactly `origin/master`, and the only modified files are the intended ones. Staging explicitly and committing now.

302. assistant (2026-07-09T05:10:57.380Z)

Staged set is clean: three intended files. Creating the progress logging commit.

303. assistant (2026-07-09T05:11:05.500Z)

The commit is made. I?m verifying it?s the only commit over `origin/master`, then I?ll push the branch for review.

304. assistant (2026-07-09T05:11:09.977Z)

Only my logging commit is on the branch, and the working tree is clean. Pushing it now.

305. assistant (2026-07-09T05:11:20.570Z)

The branch is on GitHub. Since this is a repo behavior change rather than just local cleanup, I?m opening it as a small PR so it can go through the same review path as the vault migration work.

306. assistant (2026-07-09T05:11:33.562Z)

PR #6 is open. I?m refreshing the repo handoff with this new open thread and verification results so the next session doesn?t have to rediscover it.

307. assistant (2026-07-09T05:11:51.166Z)

The handoff now records PR #6 and the checks I ran. I?ll commit that one file onto the PR branch.

308. assistant (2026-07-09T05:11:54.323Z)

Only the handoff changed. Staging and committing it separately so the code change remains easy to review.

309. assistant (2026-07-09T05:12:03.784Z)

The PR branch has the handoff commit too. I?m pushing it and then I?ll confirm the PR/working tree state.

310. assistant (2026-07-09T05:12:13.905Z)

Pushed. One last check: clean tree, PR URL, and the branch?s commits over master.

311. assistant (2026-07-09T05:12:23.968Z)

Done. I added finer-grained long-run logging and opened PR #6 for review: [Add detailed batch progress logging](https://github.com/Khelsutra/rally-corpus-vault/pull/6).

What changed:
- `PROGRESS` lines now include current GB, percent, `rate_mb_s`, `elapsed_s`, and `eta_s`.

- `BATCH_ITEM_START` / `DONE` / `FAILED` lines now include item size plus whole-batch percent.
- Runbook now documents what operators should expect in `scratch/bootstrap_golden.log`.
- Smoke/test coverage now asserts the richer fields are present.

Verified:
- `python -m pytest tests` -> 42 passed
- `python -m ruff check .` -> clean
- `python -m storagectl.storagectl smoke-test --log scratch\progress_logging_smoke.log` -> passed

Branch is pushed and clean:
- `agent/progress-percent-logging`
- Commits: `0271009` logging change, `d464997` handoff update.

312. user (2026-07-09T05:12:24.509Z)

As a follow up can you identify all the videos which now can be deleted from both F: and G (G:\My Drive\Sharing has all the videos concerned with this project) so that I can save on the redundant storage? Please show me the locations so that I can inspect and delete and also make sure to do some MD5 or checksum comparison so we are sure we are deleting an already vaulted copy

313. assistant (2026-07-09T05:12:37.447Z)

I?ll do this as an inspection/reporting pass only: verify the vault copies first, then checksum candidate files on F: and under `G:\My Drive\Sharing`, compare them to the committed vault manifest, and produce a deletion-candidate report with exact paths. No deletes.

314. assistant (2026-07-09T05:12:51.979Z)

The repo is fresh. I?m checking the current branch, manifest/config, and the existing dedup guidance so the report lines up with the vault?s conventions.

315. assistant (2026-07-09T05:13:12.579Z)

I?m going to treat ?safe to delete? narrowly: a candidate must match a vaulted `video_id` by full-file MD5 and size, and it must not be the canonical vault copy under `...\corpus\videos\full`. First I?m re-verifying the two durable vault tiers.

316. assistant (2026-07-09T05:15:21.474Z)

`status` confirms the 15-video vault shape, but the full `verify F/G` hashes are slower than the default timeout. I?m rerunning them with a longer timeout, one drive at a time, because this gate matters before recommending deletions.

317. user (2026-07-09T05:18:55.093Z)

<turn_aborted>
The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.
</turn_aborted>

318. user (2026-07-09T05:20:07.116Z)

the CI failed for the PR

319. assistant (2026-07-09T05:20:14.452Z)

Got it, switching to the PR CI failure. I?ll use the GitHub CI-fix workflow, first checking whether the interrupted local verify left any `storagectl` process running, then I?ll inspect the failing check logs for PR #6.

320. assistant (2026-07-09T05:20:23.702Z)

The workspace path I was using just came back unavailable for that process check, so I?m locating the checkout again before touching the PR.

321. assistant (2026-07-09T05:20:29.458Z)

No Python verify process is running, so the interrupted checksum pass didn?t leave a local worker behind. The prior Google Drive checkout path is not mounted right now; I?ll find the local repo copy and continue from there.

322. assistant (2026-07-09T05:20:40.458Z)

I don?t see a second local checkout under `C:\Users\avidu\Projects`. I can still inspect the PR through GitHub, then I?ll either patch once the checkout is back or clone a fresh working copy if needed.

323. assistant (2026-07-09T05:20:53.011Z)

Using the `github:gh-fix-ci` workflow for this: first auth/check metadata, then failing Actions logs. Since the local checkout is unavailable, I?ll use `gh --repo Khelsutra/rally-corpus-vault` for inspection.

324. assistant (2026-07-09T05:20:59.815Z)

The failing check is the single GitHub Actions job `ci / test` on PR #6. I?m pulling that run log now to isolate whether it?s a real behavior failure or a formatting/platform mismatch.

325. assistant (2026-07-09T05:21:15.446Z)

Root cause found: the PR code itself passed lint, but CI fails before tests during manifest schema validation. The committed `manifests/corpus_index.json` has `source_capture.gopro_original: null` for unknown metadata, while `schema/corpus_index.schema.json` currently requires that field to be a string.

326. assistant (2026-07-09T05:21:19.887Z)

CI failed in the manifest schema gate, not in the progress logging code.

Failing check: `ci / test` on PR #6
Run: https://github.com/Khelsutra/rally-corpus-vault/actions/runs/28995686802

Root cause:
```text
jsonschema.exceptions.ValidationError: None is not of type 'string'
...
source_capture.gopro_original
```

The committed manifest has unknown source metadata as `null`, but the schema requires `gopro_original` to be a string. This likely also applies to other optional `source_capture` fields like `device` / `capture_date`.

Proposed fix: update `schema/corpus_index.schema.json` so optional source-capture metadata accepts `string` or `null`, then add/adjust a schema validation test so this cannot regress. I?ll need a working checkout again; the earlier `G:\Other computers\...` checkout is currently not mounted from this shell, so I can either wait for G: to come back or clone a fresh local copy under `C:\Users\avidu\Projects` and push the fix to PR #6.

327. user (2026-07-09T05:22:15.251Z)

why do you need G:\Other Computers ?

328. assistant (2026-07-09T05:22:51.796Z)

I don?t need `G:\Other computers`.

I used it only because that was the checkout we had already been working from during the vault migration:

`G:\Other computers\My Laptop (1)\Projects\khelsutra-guru\rally-corpus-vault`

That path looks like a Google Drive mirrored ?other computer? location, so it can disappear from the shell when Drive isn?t mounted/synced. For fixing CI, that?s the wrong dependency. Any clean git checkout is enough.

Better path now: clone/use a normal local checkout under:

`C:\Users\avidu\Projects\khelsutra-guru\rally-corpus-vault`

`G:\My Drive\Sharing` is only relevant later for the duplicate-video scan/report, not for the PR CI fix.

329. user (2026-07-09T05:24:52.125Z)

ok then use the same Projects/khelsutra-guru on this local mahcine, which is what I was hoping you were using but seems you jumped to another computers drive

330. assistant (2026-07-09T05:25:05.194Z)

You?re right. I should have kept the code checkout on the local `C:\Users\avidu\Projects\khelsutra-guru` side and only treated G: as data/storage. I?ll move the repo work there now and leave the Drive paths for vault media only.

331. assistant (2026-07-09T05:25:19.423Z)

Using the `github:gh-fix-ci` flow for this PR. I?ve already isolated the failure to schema validation; now I?m going to create/use a normal local checkout under `C:\Users\avidu\Projects\khelsutra-guru`, patch PR #6 there, and push the fix.

332. assistant (2026-07-09T05:25:27.734Z)

There?s no local `rally-corpus-vault` checkout in `C:\Users\avidu\Projects\khelsutra-guru`, so I?m cloning it there now. After clone, I?ll pull/fetch and check out the PR branch from origin.

333. assistant (2026-07-09T05:25:35.954Z)

The local checkout is in place. I?m making it fresh and moving onto the PR branch from the remote before reading or editing files.

334. assistant (2026-07-09T05:25:53.576Z)

Now we?re on the PR branch in the local `C:` checkout. I?m reading the schema and manifest shape to make the smallest fix for the actual CI failure.

335. assistant (2026-07-09T05:26:02.243Z)

The mismatch is exactly as the CI log suggested: the CLI intentionally writes `None` for unknown `device`, `capture_date`, and `gopro_original`, but the schema says string-only. I?m going to make those schema fields nullable and add a test that validates the committed manifest.

336. assistant (2026-07-09T05:26:09.346Z)

I?m editing the schema and adding a small regression test now. This is a CI-only correctness fix; it doesn?t touch the progress logging behavior.

337. assistant (2026-07-09T05:26:26.028Z)

The schema now allows nulls for the metadata fields the CLI already emits, and the committed manifest is covered by a test. I?m running the same gates CI uses, plus the full suite.

338. assistant (2026-07-09T05:26:32.506Z)

Local Python didn?t have `jsonschema`, so the new test failed at collection before actually checking the fix. CI installs dev dependencies, so I?m matching that locally with `pip install -e ".[dev]"`, then rerunning the gates.

339. assistant (2026-07-09T05:26:54.148Z)

The local checkout now matches CI?s dev dependency setup. I?m rerunning lint, schema validation, full tests, and the smoke test.

340. assistant (2026-07-09T05:27:02.515Z)

All local gates pass now: lint, full tests, schema validation, and smoke-test. I?m updating the handoff to record the local checkout move and CI fix, then I?ll commit and push the PR branch.

341. assistant (2026-07-09T05:27:09.462Z)

The handoff still has the old test count and doesn?t say the repo now lives locally under `C:`. I?m making that honest before pushing.

342. assistant (2026-07-09T05:27:24.292Z)

The handoff is updated. I?m checking branch/HEAD and staging only the schema, regression test, and handoff.

343. assistant (2026-07-09T05:27:36.054Z)

Staged set is exactly the CI fix, its regression test, and the handoff note. Committing and pushing to PR #6 now.

344. assistant (2026-07-09T05:27:46.194Z)

The branch has three PR commits, all ours, and the working tree is clean. Pushing the CI fix to GitHub.

345. assistant (2026-07-09T05:27:54.235Z)

The fix is pushed. I?m watching PR #6?s check rollup now; it may take a moment for GitHub to start the new run.

346. assistant (2026-07-09T05:30:10.028Z)

CI is green now. I?ll do one last clean-tree and PR status check, then I?ll hand back the exact result.

347. assistant (2026-07-09T05:30:21.085Z)

PR #6 is already merged and CI is green. The merged handoff still says ?PR #6 is open,? so I?m going to clean that up on `master` now; it?s exactly the kind of small checkpoint the handoff is meant to capture.

348. assistant (2026-07-09T05:30:26.570Z)

I?m on local `master`; pulling the merge commit before reading or editing it.

349. assistant (2026-07-09T05:30:38.388Z)

The fast-forward pulled the merged PR, and the handoff has only that stale ?wait for PR #6? line left. I?m patching it to reflect shipped state.

350. assistant (2026-07-09T05:30:55.557Z)

Handoff now says PR #6 is merged. I?m committing only that file directly on `master`, then pushing the checkpoint.

351. assistant (2026-07-09T05:31:00.542Z)

`master` is at the PR #6 merge commit and only the handoff changed. Staging that single file.

352. assistant (2026-07-09T05:31:25.159Z)

The handoff checkpoint is pushed to `master`. I?m doing a final clean-tree check.

353. assistant (2026-07-09T05:31:32.916Z)

You were right to call that out. I moved the repo work to the local checkout:

`C:\Users\avidu\Projects\khelsutra-guru\rally-corpus-vault`

CI is fixed and PR #6 is now merged: https://github.com/Khelsutra/rally-corpus-vault/pull/6

What I fixed:
- Schema now allows `null` for optional `source_capture.device`, `capture_date`, and `gopro_original`, matching what `storagectl` writes.
- Added `tests/test_schema.py` so committed manifests validate against the schema in tests.

Verified locally:
- `python -m ruff check .` -> clean
- CI schema validation snippet -> `valid: manifests\corpus_index.json`
- `python -m pytest tests` -> 43 passed
- `python -m storagectl.storagectl smoke-test --log scratch\ci_fix_smoke.log` -> passed

GitHub CI passed, PR #6 merged, and I also pushed a handoff update on `master`:
- `76a2ed7 Update handoff after progress logging merge`

Working tree is clean.

354. user (2026-07-09T05:31:55.311Z)

can you print a summary of the vault right now

355. assistant (2026-07-09T05:32:02.719Z)

I?ll print the vault?s current summary from the local `C:` checkout and refresh it from remote first so the manifest view is current.

356. assistant (2026-07-09T05:32:11.222Z)

Repo is current. I?m running the vault status summary now.

357. assistant (2026-07-09T05:32:24.724Z)

That summary is using the portable manifest but no local tier config, so it sees 15 videos but no durable tier definitions. I?m checking whether this local clone has `vault.local.json`; if not, I?ll report the manifest summary and call out the missing local config clearly.

358. assistant (2026-07-09T05:32:29.538Z)

There?s no `vault.local.json` in the new local clone, which explains the empty tier list in the CLI status. I?ll compute the portable manifest summary directly so you still get the useful

vault state.

359. assistant (2026-07-09T05:32:40.522Z)

I have the manifest-backed summary. One note: because this `C:` clone is fresh and does not yet have the gitignored `vault.local.json`, the raw `storagectl status` command reports no configured durable tiers; the committed manifest itself still records F and G copies for all 15 videos.

360. assistant (2026-07-09T05:32:45.384Z)

Vault summary from `manifests/corpus_index.json`:

```text
Videos: 15
Golden videos: 15
Full-res corpus size: 120.1 GB

Manifest tiers:
  F: 15 full-res copies, 120.1 GB
  G: 15 full-res copies, 120.1 GB
```

Current `storagectl status` from the fresh local `C:` checkout prints:

```text
# corpus: 15 unique videos across tiers []

N+1 violations: 15
```

That warning is because this clone does not have the gitignored `vault.local.json`, so the CLI does not know which local tiers are durable. The committed manifest still records all 15 videos on both `F` and `G`.

The 15 vaulted videos are:
`mahadevpura_singles`, `adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`, `testlarge_short`, `mahadevpura_2`, `GX010128`, `mahadevpura_1`, `GX030094`, `Badminton_BXH_2`, `GX010141`, `GX010137`, `Boxhill_Doubles`, `GX020094`.

361. user (2026-07-09T05:32:45.852Z)

also can you find the duplicate videos from the vault in the F: drive for now?

362. assistant (2026-07-09T05:33:01.041Z)

Yes. I?ll do a read-only F: pass: pull the repo, load the committed vault manifest, enumerate F: video files by size first, then MD5 only size-matching candidates. I?ll exclude the canonical vault copy under `F:\...\corpus\videos\full` from ?delete candidate? and report only extra matching paths.

363. assistant (2026-07-09T05:33:22.235Z)

I?m going to scan the whole `F:\` tree, but cheaply: only files whose byte size equals one of the 15 vaulted masters get full MD5. That keeps the checksum proof real without hashing unrelated media.

364. assistant (2026-07-09T05:33:41.227Z)

F: is mounted and the canonical vault folder exists. I?m starting the checksum scan now; it will print progress as it enumerates candidates and hashes matching-size files.

365. assistant (2026-07-09T05:34:13.738Z)

Tiny scripting snag: Python raw strings cannot end with a single backslash. No scan ran. I?m rerunning with forward-slash Windows paths.